DATA305 / DATA475 · Machine Learning Methods

# Keras API Deep Dive

*Functional API · Callbacks · Model Persistence*

**Lecture 3**

Week 5 · Google Colab ready

*Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington*

# Today's Agenda



*Five topics*

## 01 Sequential API Limitations

*When a linear stack is not enough*

## 03 Multi-Output Models

*Named heads, per-output losses, loss weights*

## 02 Keras Functional API

*Input(), layer chaining, Model() — branching and merging graphs*

## 04 Callbacks — Automate Training

*EarlyStopping · ModelCheckpoint · ReduceLROnPlateau*

## 05 Model Persistence

*save() · load_model() · what gets saved*

# Lecture 2 → Lecture 3



**In L2** we built a complete MLP with Sequential API and trained it on Fashion MNIST. **Today** we master the full Keras workflow, starting with **when Sequential is not enough**, and ending with saving trained models for reuse.

## Lecture 2 — Covered

- MLP architecture and forward pass
- Activation functions: Sigmoid, Tanh, ReLU, Softmax
- Loss functions: MSE, BCE, Sparse CE
- Sequential API: build → compile → fit → evaluate
- Fashion MNIST demo and learning curves

## Lecture 3 — Today

- Why Sequential has limits: no branching, no multi-in/out
- Functional API: Input() → layer chaining → Model()
- Wide & Deep as the motivating architecture
- Multi-output models with per-output loss weights
- Callbacks: EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
- Saving and reloading trained models

# Sequential API — What It Cannot Do



### Sequential API — Good for

- Linear chain: input → hidden → hidden → output
- Simple classifiers and regressors (L2 Fashion MNIST)
- Minimal code, easy to read

### Sequential API — Cannot express

- Multiple inputs (e.g., image + text)
- Multiple outputs (predict two targets at once)
- Branching / merging paths
- Skip connections / residual blocks
- Shared layers across branches

**Three scenarios that need Functional API:**

### Multi-Input

Two Input() objects, processed separately, merged with Concatenate() before the output.

### Skip / Residual Connection

Layer k+1 output also feeds layer k+3, bypassing k+2. Used in ResNet (L9).

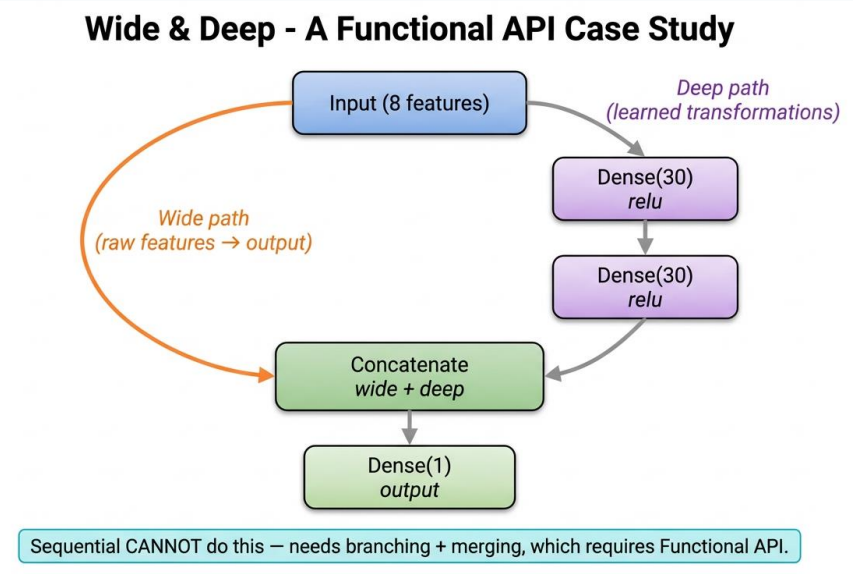
### Wide & Deep ← today's example

Short path: raw features → output directly. Long path: features → dense layers → output.

**Sequential** = linear stack · **Functional** = branching and merging model graphs · **Subclassing** = advanced custom modelling; not needed in this course

# Wide & Deep — A Concrete Functional API Example






python · Wide & Deep Functional

```python
inp = tf.keras.Input(shape=(8,))

# Deep path
x = Dense(30, 'relu')(inp)
x = Dense(30, 'relu')(x)

# Wide path: raw input skips
# deep layers → concat
concat = Concatenate()([inp, x])

out = Dense(1)(concat)

model = tf.keras.Model(
    inputs=inp,
    outputs=out
)

# Sequential CANNOT express
# branching + merging
```

## Why this matters

Short path: learns simple linear relationships.
Long path: learns complex non-linear patterns.
Both contribute — the model does both simultaneously.

# Functional API — Build the Graph



### Step 1

**Define Input**

```
inp = Input(shape=(8,))
```

### Step 2

**Chain layers like functions**

```
x = Dense(64, "relu")(inp)
x = Dense(64, "relu")(x)
```

### Step 3

**Branch outputs**

```
out = Dense(1, name="main")(x)
aux = Dense(1, name="aux")(x)
```

### Step 4

**Create Model**

```
model = Model(
    inputs=inp,
    outputs=[out, aux])
```

python · Google Colab

```python
# Data already loaded: X.shape = (20640, 8)  |  y.shape = (20640,)

# Step 1: symbolic input
inp = tf.keras.Input(shape=(8,))

# Steps 2–3: chain layers → define output
x   = tf.keras.layers.Dense(64, activation='relu')(inp)
x   = tf.keras.layers.Dense(64, activation='relu')(x)
out = tf.keras.layers.Dense(1)(x)   # regression, no activation

# Step 4: finalize graph
model = tf.keras.Model(inputs=inp, outputs=out)
model.summary()
```

### Symbolic

Input() = placeholder, not data. Graph is built first; computation happens in fit().

### Reusable layers

Call same layer on two tensors → weight sharing (Siamese nets). Impossible with Sequential.

### Model validates

Model(inputs=, outputs=) checks shapes and enables summary() before any training.

# Functional API — Compile, Fit, Evaluate, Predict



python · compile + fit + evaluate

```python
# (X_test, y_test, X_new assumed from train/test split)

# Compile — identical to Sequential
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['RootMeanSquaredError']
)

# Fit
history = model.fit(
    X, y,
    epochs=50,
    validation_split=0.1
)

# Evaluate on test set
loss, rmse = model.evaluate(X_test, y_test)
print(f'Test RMSE: {rmse:.3f}')

# Predict
y_pred = model.predict(X_new)
```

### Once built → identical to Sequential

compile(), fit(), evaluate(), predict() all work exactly the same way. The Functional API only changes how you build the graph — the training loop is unchanged.

### California Housing

- 20 640 samples, 8 numeric features
- Regression: predict house value
- No GPU needed — trains in seconds
- sklearn: fetch_california_housing()

# QUICK QUIZ

You want a model that takes two separate inputs — a vector of 20 numeric features AND a one-hot label of size 10 — processes them through separate Dense layers, then concatenates and predicts one output.

Which Keras API should you use, and why?

**A** Sequential — concatenate the two inputs first, then pass one combined tensor

**B** Functional — two separate Input() objects, merged with Concatenate()

**C** Either works — pass a list of inputs to Sequential's fit()

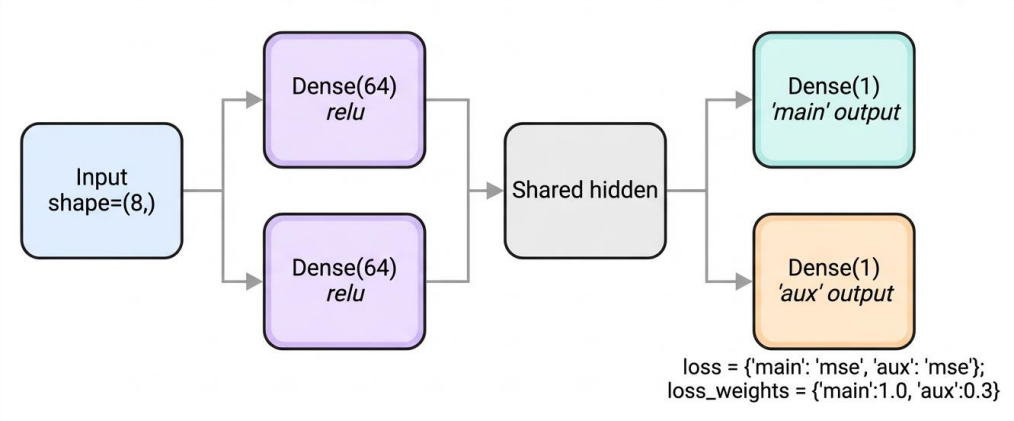
**D** Subclassing API only — Functional does not support two inputs

*Think about what Sequential requires each layer to have.*

# Multi-Output Models — Per-Output Loss Weights



Multi-Output Model – Two predictions, one forward pass




python · multi-output

```python
inp = tf.keras.Input(shape=(8,))
x = Dense(64, 'relu')(inp)
x = Dense(64, 'relu')(x)

# Two output heads
out_main = Dense(1, name='main')(x)
out_aux  = Dense(1, name='aux')(x)

model = tf.keras.Model(
    inputs=inp,
    outputs=[out_main, out_aux]
)
model.compile(
    optimizer='adam',
    loss={'main':'mse','aux':'mse'},
    loss_weights={'main':1.0,'aux':0.3}
)
# Total = 1.0*L_main + 0.3*L_aux
```

### Why an auxiliary output?

- Auxiliary head = extra training signal for shared layers.
- Can sometimes improve the main task.
- Use a smaller loss weight so it does not dominate.

# Callbacks — What They Are and the Three We Focus On



A Callback is called by Keras at specific training events. Pass a list via `model.fit(..., callbacks=[cb1, cb2])`. Three callbacks cover almost every use case:

## EarlyStopping

**When to use:**

Val metric stops improving

```
patience=5, restore_best_weights=True
```

Stops when val metric stops improving; restores best weights.

## ModelCheckpoint

**When to use:**

Save model during training

```
save_best_only=True, 'best.keras'
```

Saves the best model to disk whenever val_loss improves.

## ReduceLROnPlateau

**When to use:**

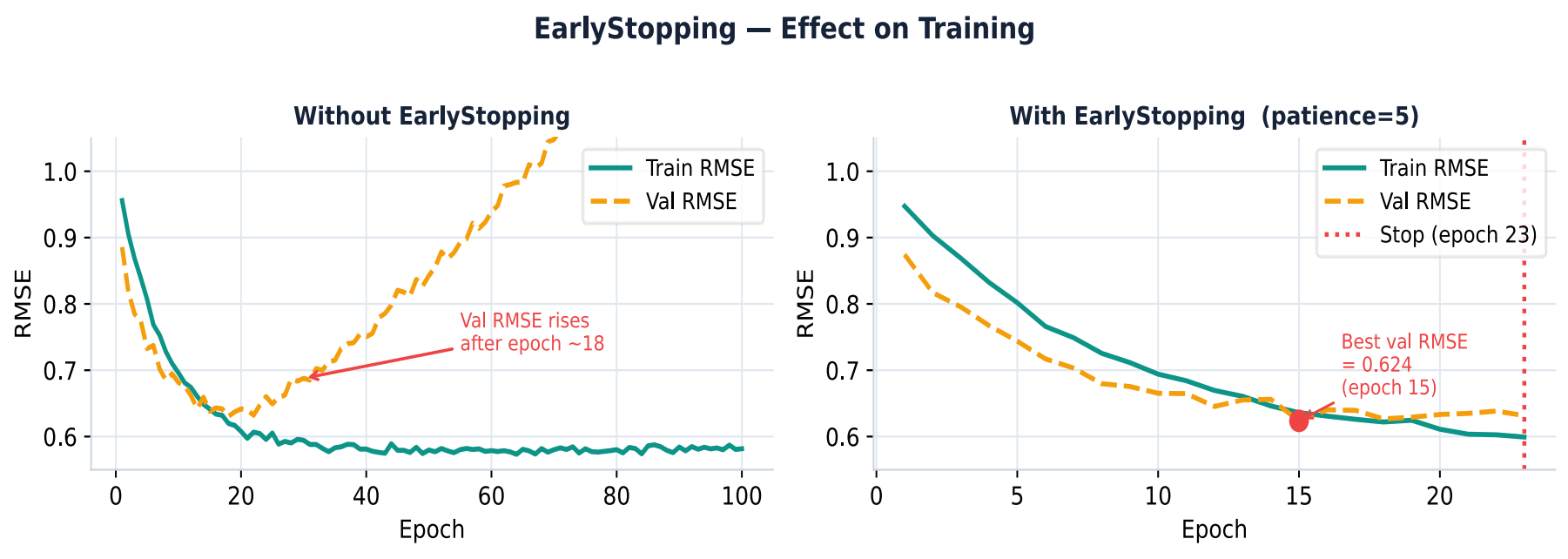
LR too large to converge

```
factor=0.5, patience=3, min_lr=1e-6
```

Reduces LR by factor when val_loss stops improving.

# EarlyStopping — Stop Before Overfitting






python · EarlyStopping

```python
es = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=5,
    restore_best_weights=True)   # strongly recommended
model.fit(X, y, epochs=200, callbacks=[es])
```

**patience**

Epochs to wait for improvement before stopping.

**restore_best_weights**

Rolls back to the best epoch's weights at the end.

**epochs=200**

Set large — EarlyStopping will find the stop point.

# ModelCheckpoint and ReduceLROnPlateau



python · combining callbacks

```python
# Save only when validation improves
ckpt = tf.keras.callbacks.ModelCheckpoint(
    'best.keras',
    monitor='val_loss', save_best_only=True)

# Halve LR after 3 epochs of no improvement
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5, patience=3, min_lr=1e-6)

# Combine in fit()
model.fit(X_tr, y_tr, epochs=200,
    validation_split=0.1,
    callbacks=[es, ckpt, reduce_lr])
```

## ModelCheckpoint

- Saves to disk whenever val_loss improves (save_best_only=True)
- With the default full-model .keras save, stores arch + weights + optimizer state
- Protects against crashes — resume from last checkpoint

## ReduceLROnPlateau

- Lowers LR when val_loss plateaus for patience epochs
- Avoids oscillating around a minimum with too-large steps
- min_lr sets a floor — LR never shrinks below it

# QUICK QUIZ

You train with `EarlyStopping(patience=5, restore_best_weights=True)`.

Validation loss improves up to epoch 18, then gets worse for 5 consecutive epochs. Training stops at epoch 23.

What are the model weights after training ends?

**A** Weights from epoch 23 — the last epoch before stopping

**B** Weights from epoch 18 — the best validation epoch, restored automatically

**C** Random initialization — EarlyStopping resets weights on stop

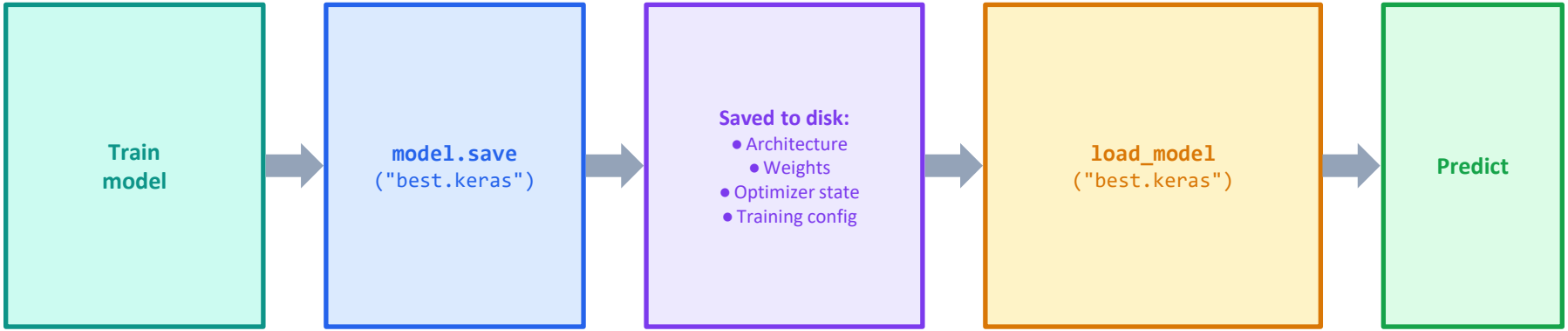
**D** Weights from epoch 1 — EarlyStopping always restores the start state

*Hint: what does restore_best_weights=True actually do?*

# Model Persistence — Save, Load, Predict



*Keras Native Format (.keras) | includes everything needed for inference*




python · save and reload

```python
# Save (recommended: .keras format)
model.save('best.keras')

# Reload in any Python session
loaded = tf.keras.models.load_model('best.keras')

# Predict immediately — no recompile
y_pred = loaded.predict(X_new)
```

| Saved in .keras |
| --- |
| Architecture (layer types, connections) |
| Weight values (trainable + non-trainable) |
| Optimizer state (step, momentum buffers) |
| Training configuration (loss, metrics) |
| *NOT saved: history object* |

# Full Pipeline — Reference Slide



REFERENCE ONLY — complete workflow in one place. Use during practice sessions, not for line-by-line walkthrough.

python · complete pipeline

```python
# [Data loading + split omitted for brevity]

# Functional model (multi-output)
inp      = tf.keras.Input(shape=(8,))
x        = tf.keras.layers.Dense(64,'relu')(inp)
x        = tf.keras.layers.Dense(64,'relu')(x)
out_main = tf.keras.layers.Dense(1,name='main')(x)
out_aux  = tf.keras.layers.Dense(1,name='aux')(x)
model    = tf.keras.Model(inp,[out_main,out_aux])

model.compile(optimizer='adam',
    loss={'main':'mse','aux':'mse'},
    loss_weights={'main':1.0,'aux':0.3})

es   = tf.keras.callbacks.EarlyStopping(
    patience=5,restore_best_weights=True)
ckpt = tf.keras.callbacks.ModelCheckpoint(
    'best.keras',save_best_only=True)

model.fit(X_tr,{'main':y_tr,'aux':y_tr},
    epochs=100,validation_split=0.1,
    callbacks=[es,ckpt])

model.save('housing_final.keras')
```

**Multi-output graph**

Two named heads from shared layers.

**Dict compile()**

loss and loss_weights matching output names.

**EarlyStopping + Checkpoint**

Auto-stops + restores + saves to disk.

**model.save()**

Always before closing Colab session.

# QUICK QUIZ

You save a model with `model.save('housing.keras')`.

Your colleague opens a fresh Python session and runs:

```
loaded = tf.keras.models.load_model('housing.keras')
y_pred = loaded.predict(X_new)
```

What does the loaded model know — and what does it NOT know?

**A** Knows everything: architecture, weights, optimizer state, and history object

**B** Knows architecture and weights, but NOT optimizer state or history

**C** Knows architecture, weights, AND optimizer state — but NOT the history object

**D** Only knows architecture — weights must be loaded separately

# Getting Started — Before Tutorial 1



## 1 Run the Functional API demo in Google Colab

*Open Google Colab → paste code from slides 6–7 → run · Note which epoch EarlyStopping stops at*

## 2 Experiment — change patience

*Try patience=1, 5, 20 · Does a longer patience improve test RMSE? · Compare final epoch counts*

## 3 Add ReduceLROnPlateau and read the loss curve

*Add ReduceLROnPlateau(patience=3, factor=0.5) to callbacks · Plot history['main_loss'] · Do you see the LR drops?*

*Reading: Géron Ch. 10 — Functional API (pp. 329–335), Callbacks (pp. 338–340).*

# Lecture 3 Summary

*Key Takeaways*

Sequential = linear stack only. Need branching or multi-in/out? → Functional API.

Functional: inp = Input(shape=...) → chain layers → Model(inputs=, outputs=).

Multi-output: named heads + loss dict + loss_weights. Aux head provides an extra training signal to shared layers.

EarlyStopping(patience=5, restore_best_weights=True) — set epochs large and let it stop.

.keras saves arch + weights + optimizer state. history object is NOT saved.

A practical default workflow: Functional + EarlyStopping + ModelCheckpoint + model.save().

*Next up: Lecture 4 — Training Deep Networks I · Gradients · Initialization · Batch Normalization*